

# /v2/ — what was built and how it's put together

Written 19 August 2026, for Louis, covering the whole `index2.html` effort: the initial build (Opus 5 planned it, Sonnet 5 built it) and every fix since. Companion to `v2Handover.md` (that one's for Damian, short, framed against his prototype). This one's longer on purpose — enough to edit and extend the code from, not just read once.

## 1. What this is

`index.html` (Damian's) still renders at `/`, completely untouched. `index2.html` renders at `/v2/`, built from the same design intent — the drawio layout diagram and `docs/claude/design-plan.html` — but restructured so the future infinite-scroll loader (Issue #7) has a real contract to build against instead of duplicated markup. The two pages exist side by side deliberately; nothing has been deleted or merged yet.

## 2. Timeline of this session's work

Roughly in order:

1. **Initial build.** New templates, fixture-backed views, the CSS component set, `road.js` with a mobile disclosure drawer. Committed as 7090267.
2. **Masthead fix.** Title was left-aligned and small; centred it and roughly doubled its size; moved "Jump to the end of the road" into its own row underneath rather than crowding the title row.
3. **Mobile grid bug.** Below 640px, artwork and the head tile were landing in a phantom second grid column instead of the single mobile column — three symptoms (detached images, misplaced head tile, blank space on expand) turned out to be one root cause.
4. **Row-height / overflow rework.** At desktop widths, a long description could grow a row taller than its own artwork, breaking the chain. Fixed by pinning row height strictly to the artwork's rendered height and giving `.road-meta` a flex layout where only the description clips.
5. **Mobile drawer polish.** Fixed a flash-of-expanded-then-collapsed on load/resize, a persistent gap on collapsed mobile sections, and missing horizontal padding on expanded mobile text.
6. **Description overflow toggle,** several iterations: an overlay-expansion design was tried and abandoned (see §6.4 — content-visibility blocked it outright), replaced with an internally-scrolling fixed-size box; a "Show less"

button that scrolled away with the text got pinned properly by splitting the scrolling element out; the toggle was then moved out of the description box entirely into the Share/Report action row, because that was the only fix that solved both "text pokes out past the button" and "toggle eats into reading space" at once.

7. **Wrap-order fix.** When Share/Report/More wrap to two lines at narrow desktop widths, More/Less now always lands on its own line *above* Share+Report (not below), because Share+Report are grouped as one atomic flex item that can't split.
8. **Road width.** Max width of the artwork column raised ~20%, so it keeps growing on large monitors instead of plateauing around 1300px viewport width.
9. **Accent colour.** Swapped from green to a deep ink-blue (light) / soft sky-blue (dark), same role everywhere — numeral hover, focus rings, link hovers.
10. **Handover note for Damian** written to docs/notes/v2Handover.md.

Steps 2–9 were all done after the initial commit; you (Louis) committed them yourself as 06e80e3, plus the blue accent change is the one thing still uncommitted as of writing this.

### 3. File structure

```
aLongRoad/  
  render/  
    templates/render/  
      index2.html      full page: masthead, head tile, road, sentinel/stat  
      _batch.html      loop over one batch → sections/slots, then cursor  
      _section.html    ONE live section – reused by first load and every  
      _slot.html       vacant / reserved placeholder tile  
      _road_end.html   "The End" – only ever included once the batch has  
    static/render/js/  
      road.js          mobile drawer + desktop description toggle (loader  
    fixtures.py         stand-in for the not-yet-built model (Issue #3)  
    views.py            index2_view, road_batch_view, index_view (Damian'  
    urls.py             ' ' → index, 'v2/' → index2, 'v2/road/' → road_ba  
  theme/  
    templates/base.html shared shell both index.html and index2.html ext  
    static_src/src/  
      tokens.css       palette, fonts, type scale, --road-w  
      base.css          reset, focus-visible, skip-link, .no-transition  
      components/  
        masthead.css   the spine grid, rails, numeral, head tile, term  
        road.css        metadata layout, drawer, description clip/togg  
        meta.css        vacant/reserved tile styling  
        slot.css  
docs/  
  notes/
```

v2Handover.md  
v2Structure.md

short note for Damian  
this file

Everything under `templates/render/` and `static/render/` is namespaced under `render/` deliberately — Django pools every app's `templates/` directory into one flat search space, so an unnamespaced `templates/index2.html` could be silently shadowed by another app. `index.html` predates this convention and is the one exception, left alone on purpose (`design-plan.html` §9.6).

## 4. The backend: fixtures, views, the loader contract

`fixtures.py` holds `_SECTIONS`, a hand-authored list of 14 dicts (`slot_number`, `state`: `live`/`vacant`/`reserved`, `title`, `author_name`, `description`, `image`, and an `anchors` tuple of four fractions). It exposes three functions `views.py` calls instead of an ORM query:

- **`get_batch(before=None, limit=5)`** → (`sections`, `next_cursor`). Pool is every slot number sorted highest-first; `before` filters to slots strictly less than it. `next_cursor` is the lowest slot number just returned, or `None` once slot 1 is included — that `None` is what stops the chain.
- **`top_section()`** — the highest live section, feeds the head tile's connecting stub.
- **`jump_to_end_cursor()`** — feeds the masthead's "jump to the end" link.

`views.py` has two view functions built on this:

```
def index2_view(request):
    before = _parse_before(request)
    sections, next_cursor = fixtures.get_batch(before=before)
    return render(request, 'render/index2.html', {...})
```

```
def road_batch_view(request):
    before = _parse_before(request)
    sections, next_cursor = fixtures.get_batch(before=before)
    return render(request, 'render/_batch.html', {...})
```

Both call `get_batch` and both render output that ends up going through `_batch.html` — `index2_view` includes it inline as part of the full page, `road_batch_view` renders *only* that fragment, at `/v2/road/?before=<slot>`. That second endpoint doesn't do anything yet — nothing calls it — but it's a real, working, fully-migration-ready fragment endpoint for Issue #7's loader to call, returning exactly the same markup a page load would have produced.

**Why "The End" can't appear early:** `_batch.html` renders sections, then checks `next_cursor`. If it's not `None`, it emits a `<li class="road-cursor" data-before="{ { next_cursor } }">` with a plain link (the no-JS fallback, and the value the future loader will read via `data-before`). If it is `None`, it includes

`_road_end.html` instead — never both, and there's no `has_more` boolean anywhere to get out of sync. The end card is physically absent from the DOM until the batch that actually reaches slot 1.

## 5. HTML / template structure

### Page composition

`index2.html` extends `base.html` (three blocks filled: `skip_link`, `font_preloads`, `body_content`) and lays out:

```
<header class="masthead">                                title + "jump to the end"
<main id="road" class="road-stack">
  <a class="road-head">                                    "add a new section" tile
  <ol class="road" reversed>
    {% include "_batch.html" %}                            sections + slots + cursor/end
  <div class="road-sentinel">                                future IntersectionObserver target
  <p class="road-status" role="status">                    future batch announcements
<footer class="road-foot"></footer>                        empty on purpose – bible §15.11
```

### The spine grid — the core idea

Every section — live or placeholder — is an `<li>` in the same `<ol class="road">`, and every one is its own CSS grid with **identical** column tracks:

```
.road-section {
  display: grid;
  grid-template-columns: minmax(0, 1fr) var(--road-w) minmax(0, 1fr);
}
```

Column 1 = numeral (flex), column 2 = artwork (fixed at `--road-w`), column 3 = metadata (flex). Because every `<li>` reads the same variable rather than computing its own percentage split, the artwork column lines up to the pixel down the whole page — that's the entire trick behind the road reading as one continuous strip. Two hairline rules, drawn once as `.road-stack::before/:after` at `calc(50% ± var(--road-w) / 2)`, are the visible edges of the road; they are not a border per section.

At desktop widths (`width >= 640px`), the row's height is pinned to the artwork's own rendered height:

```
.road-section { grid-template-rows: calc(var(--road-w) * var(--ar-num, 1))
```

`--ar-num` is the numeric aspect ratio (`width / height`), passed in per-section as an inline style. Nothing else may stretch a row past this — see §6.2 for how `.road-meta` respects that.

Below 640px the grid collapses to a single column (`grid-template-columns: minmax(0, 1fr)`), and every child that was pinned to column 2 (`.road-art`, `.road-head-art`, `.road-slot-art`) needs a matching grid-column: 1 override at that breakpoint — without it, the browser invents an implicit second column to satisfy the old pin, and that element detaches sideways. This bit the mobile layout once already (§2 item 3) and is the thing most likely to bite again if a new grid child is added without the matching override.

## One live section's anatomy

```
<li class="road-section" value="{{ s.slot_number }}" data-slot="{{ s.slot_
  data-entry-l="..." data-entry-r="..." data-exit-l="..." data-exit-r="..."
  style="--ar: {{ s.width }} / {{ s.height }}; --ar-num: {{ s.ratio }};"
  <span class="road-index" aria-hidden="true">#0007</span>
  <figure class="road-art">
    
    <button class="road-disclosure-trigger" aria-controls="drawer-7">...</button>
  </figure>
  <figcaption class="road-meta">
    <div class="road-drawer" id="drawer-7">
      <div class="road-drawer-collapse">
        <div class="road-drawer-inner">
          <h2 class="road-title">...</h2>
          <p class="road-author">By ...</p>
          <div class="road-meta-clip" id="desc-7">
            <div class="road-meta-clip-scroll"><p class="road-desc">...</p></div>
          </div>
          <ul class="road-actions">
            <li class="road-actions-group"><a href="#">Share</a><button>Re
            <li class="road-more-item" hidden>
              <button class="road-meta-more" aria-controls="desc-7"><span>
            </li>
          </ul>
        </div>
      </div>
    </div>
  </figcaption>
</li>
```

Three nested wrappers around the drawer content (`.road-drawer` → `.road-drawer-collapse` → `.road-drawer-inner`) look like more than you'd expect for one collapsible box — they exist because a CSS grid track that's animated down to `0fr` will still be held open by any padding on the item occupying that track, even though it happily shrinks the item's *content* to zero. `.road-drawer-collapse` is the actual `0fr/1fr` grid item and carries no padding; `.road-drawer-inner`, one level further in, carries the padding, and gets clipped away along with everything else. This is exactly the kind of thing that looks like accidental over-nesting until you try removing a layer and the mobile gap comes back.

`.road-meta-clip / .road-meta-clip-scroll` is a similar two-layer split, for an unrelated reason: the outer element never scrolls (so an absolutely- or statically-positioned sibling button can stay visually anchored to it), and the inner one is the only thing that ever gets `overflow-y: auto`. See §6.3.

## The chain: anchors and placeholders

`data-entry-l/r` and `data-exit-l/r` are fractions (0–1) marking where a section's own artwork meets its neighbours horizontally. `_slot.html` — used for vacant and reserved states — draws its own connecting line as an inline SVG (`viewBox="0 0 1 1", preserveAspectRatio="none"`) from its entry anchors down to its exit anchors, so a gap in the actual artwork still reads as continuous road. The head tile (`.road-head`) does the same thing in reverse, stubbing a line down from its own fixed points to the top live section's entry anchors. Fixture slot 9 exists specifically to exercise this — `Road_1.png` and `Road_2.png` only chain cleanly in one direction, so slot 9 is a deliberate vacant seam bridging the real mismatch (see the comment at the top of `fixtures.py`).

## 6. CSS architecture

### 6.1 Tokens (`tokens.css`)

One `@theme` block defines the palette as `--color-*` (published as Tailwind utilities too, e.g. `bg-paper`), then `:root` re-exports each as a plain `--name` custom property (`--paper`, `--ink`, `--accent`, ...) that every component file actually reads. The palette is overridden twice for dark mode — once under `@media (prefers-color-scheme: dark)` scoped to `:root:not([data-theme="light"])`, and once under `:root[data-theme="dark"]` for an explicit user choice to win regardless of system setting. Both blocks currently carry the same values; that duplication is intentional, not a bug — the three-state light/dark/system control (not yet built) is what will actually flip `data-theme`, and both paths need to already resolve correctly.

`--road-w: clamp(17.5rem, 42vw, 40.8rem)` is the one token that controls the whole page's scale. Raise the max clamp value to widen the road further; the 42vw slope and 17.5rem floor control narrower behaviour and weren't touched this session.

### 6.2 The row-height contract (`road.css + meta.css`)

This is the mechanism most worth understanding before editing metadata layout. The row height is a **definite** length (`$5, calc(var(--road-w) * var(--ar-num))`), and `.road-meta` at desktop widths is a flex column filling that fixed height:

```
.road-meta { height: 100%; display: flex; flex-direction: column; }
.road-title, .road-author, .road-actions { flex: 0 0 auto; }
.road-meta-clip { flex: 0 1 auto; min-height: 0; }
```

Title/author/actions take exactly their content size and never shrink. `.road-meta-clip` — the description — is the only flexible one: `flex: 0 1 auto` (no grow, can shrink) plus `min-height: 0` (flex items default to refusing to shrink below their content's natural size; this opts out of that, which is what actually lets overflow: hidden bite). If a description is short, the clip just takes its natural height and leaves whitespace above the actions row; if it's long, it's clamped to whatever's left. Nothing here can ever grow the row past the artwork's height — that's the whole point.

### 6.3 Description overflow: clip, scroll, toggle

Three cooperating pieces, all in `meta.css`, all at width  $\geq 640\text{px}$ :

- `.road-meta-clip-scroll { height: 100%; overflow: hidden; }` — clipped by default.
- `.road-meta-clip[data-expanded] .road-meta-clip-scroll { overflow-y: auto; }` — becomes internally scrollable once expanded. The box's own footprint never changes size in either state, which is what keeps the chain intact through an expand/collapse.
- A mask-image fade on the clipped tail, shown only when `[data-truncated]` is set (by `road.js`, based on actual measured overflow, not assumed).

The toggle button (`.road-meta-more`) lives in `.road-actions`, not inside the clip — see §2 item 6 for why. It's paired to its clip via `aria-controls/id` rather than DOM containment, because they're no longer ancestor/descendant.

### 6.4 Why the overlay-expand design was abandoned

Worth recording because it'll look tempting to redo: an earlier version let the expanded description grow past the row and overlap the *next* section, absolutely positioned with a high `z-index`. It doesn't work, for a real reason rather than a styling detail — `.road-section` uses `content-visibility: auto` so hundreds of offscreen sections cost nothing to render (this is a required feature, not incidental), and `content-visibility: auto` applies paint containment *unconditionally*, clipping descendant overflow to the section's own box even when the overflowing descendant explicitly says `overflow: visible`. Forcing `content-visibility: visible` to test it did let content escape — but then the *next* section's own content and the escaped overlay had unpredictable, illegible stacking order against each other, because they're separate stacking contexts with no reliable way to order them from a descendant's `z-index` alone. Internal scrolling avoids the problem entirely rather than fighting it.

## 6.5 The mobile drawer collapse

Progressive enhancement, not a default: the drawer is open in plain CSS (`grid-template-rows: 1fr`) so with JS disabled every section is fully readable at every width — this is the actual content-parity requirement (bible §11), not an edge case. `road.js` adds `[data-enhanced]` to `.road` once it's wired up buttons, and *only then* does the collapsed rule apply:

```
.road[data-enhanced] .road-section:not([data-expanded]) .road-drawer {  
  grid-template-rows: 0fr;  
}
```

A `matchMedia('(max-width: 639.98px)')` listener in `road.js` adds/removes this wiring live at the breakpoint, so `aria-expanded` is never exposed on a control that would do nothing (desktop has no accordion). The very first collapse — on load, or on crossing the breakpoint — is wrapped in a `withoutTransition()` helper (base.css's `.no-transition` utility class + a forced reflow) so it doesn't visibly flash open-then-shut.

## 6.6 The atomic wrap group

`.road-actions` is `display: flex; flex-wrap: wrap`. Share and Report are nested inside one shared `<li class="road-actions-group">` (an inner `display: flex, no wrap`) so they can never split from each other. `.road-more-item` carries `order: -1`. Result: when all three fit, order doesn't matter, one line. When they don't, the flex algorithm can only produce one outcome — More/Less alone on the first line, the atomic Share+Report pair on the second — never a stranded third button. No magic breakpoint number anywhere; it falls out of the two-items-only structure.

## 7. JavaScript (road.js)

One file, two independent, self-contained functions, both called once at the bottom on `.road` if it exists. Neither knows about the other.

- **initDisclosure(road)** — the mobile accordion. Tracks a `matchMedia` for the 640px boundary; `enhance()/unenhance()` add/remove `[data-enhanced]` and wipe expanded state; a click handler collapses whatever's open before opening the new one (true accordion, one at a time).
- **initDescOverflow(road)** — the desktop More/Less system. Builds pairs by matching every `.road-meta-more` to the `.road-meta-clip` named in its `aria-controls`. `measure()` reads every clip's scroll overflow first, *then* writes `[data-truncated]/item.hidden` for all of them — batched to avoid interleaving layout reads and writes across many sections at once. Runs on load, on a debounced resize, and on crossing the 640px boundary (`reset()`)

on the way down to mobile, since the whole block is visible there and nothing needs a toggle).

Both share `debounce()` and `withoutTransition()` from the top of the file. The loader (Issue #7) is meant to land in this same file — the sentinel and status region it will use are already sitting in `index2.html`, unused.

## 8. What's deliberately not built yet

The infinite-scroll loader itself, self-hosted fonts (tokens already read `var(--font-*)` everywhere, so this is a font-file swap, not a template change), the light/dark/system theme *control* (the no-flash script and tokens are ready, there's just no UI toggle), unloading of offscreen sections, and the report dialog. All are named, in order, in `design-plan.html`'s phase list — nothing here was skipped by accident.

One live gap: `project-bible.md / design-plan.html` were updated when `index2.html` first landed, but haven't been revised for anything since the masthead fix — worth a pass before `/v2/` replaces `/`.